

GRAN PRIX CAPYTOWN

INTEGRANTES:

AGUILAR CONTRERAS Angel Jesus

CABALLERO SALAZAR Mattias Lincoln

NECIOSUP SAAVEDRA Leslie Jazmin

TICONA SANCHEZ Camila Danna

Grupo 1 – Sección 002

Facultad de Ingeniería, Universidad ESAN Lima, Perú



EL RETO DEL LABERINTO

Navegar de A4 a F1 en una pista 6×4 de celdas de 60 cm, sin conocimiento previo del recorrido a través de un mapeo usando el Lidar MS200 en MicroRos Car.

- 01

Evitar contactos

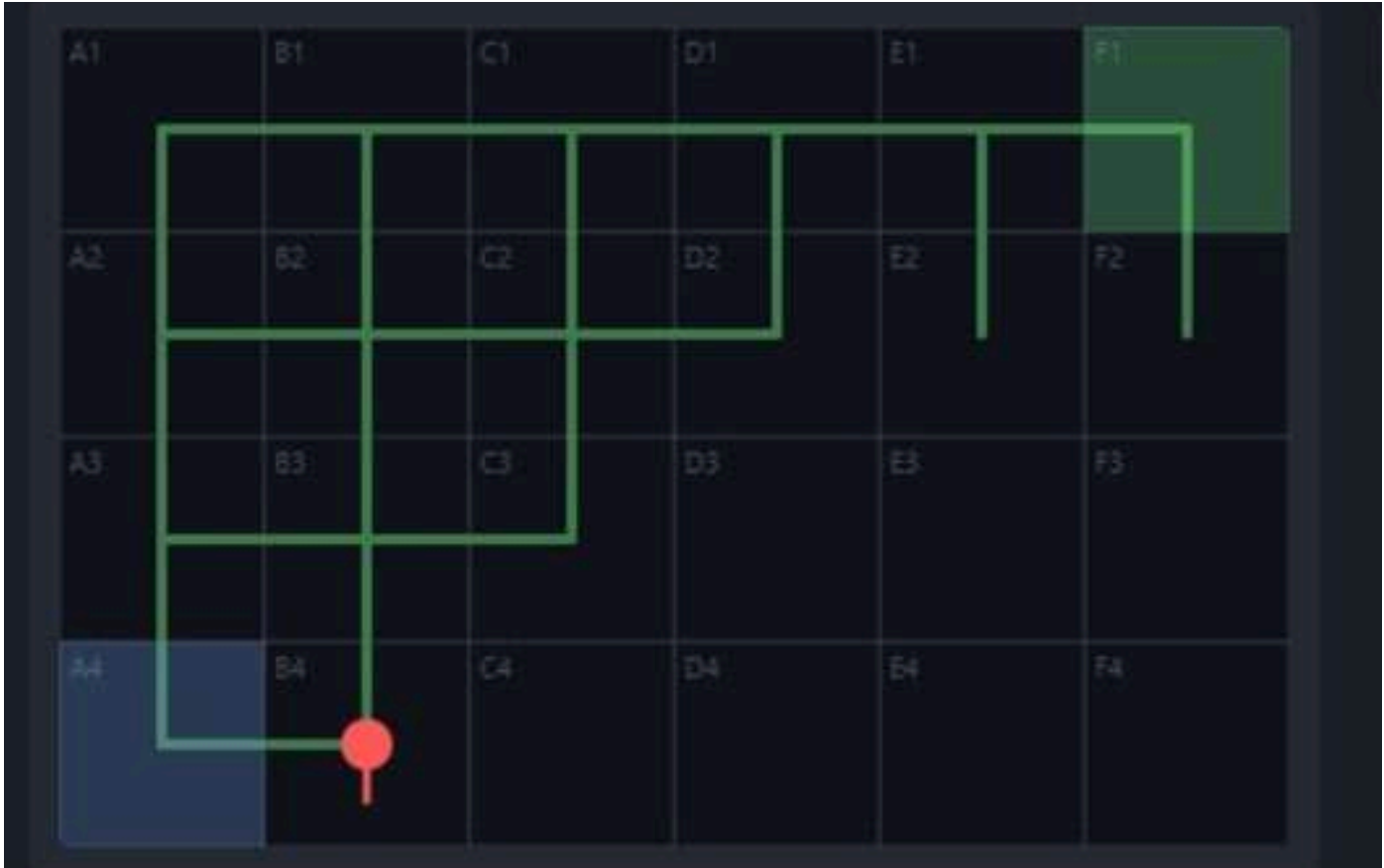
LiDAR como referencia geométrica y seguridad frontal/lateral.
- 02

Respetar PARE

Cámara detecta señales rojas y fuerza pausa completa de 3 s.
- 03

Optimizar ruta

Primera ronda explora; segunda usa mapa y BFS.



CARACTERÍSTICAS PRINCIPALES DEL ESCENARIO

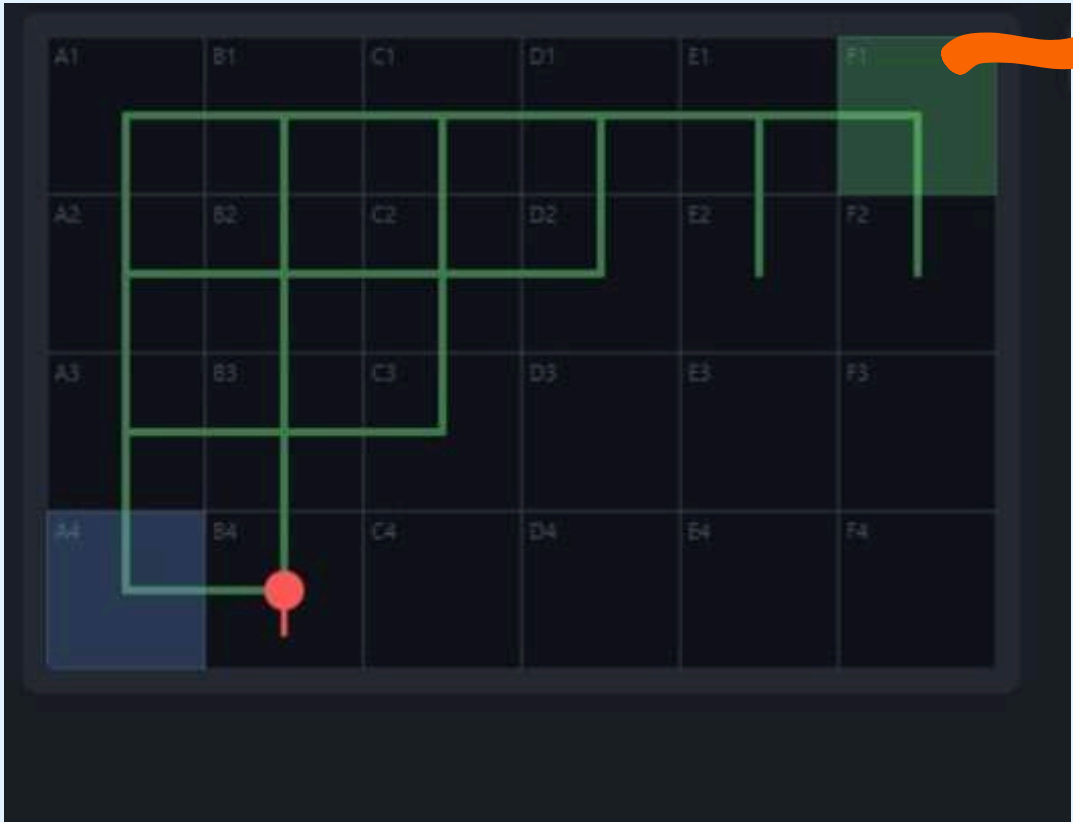
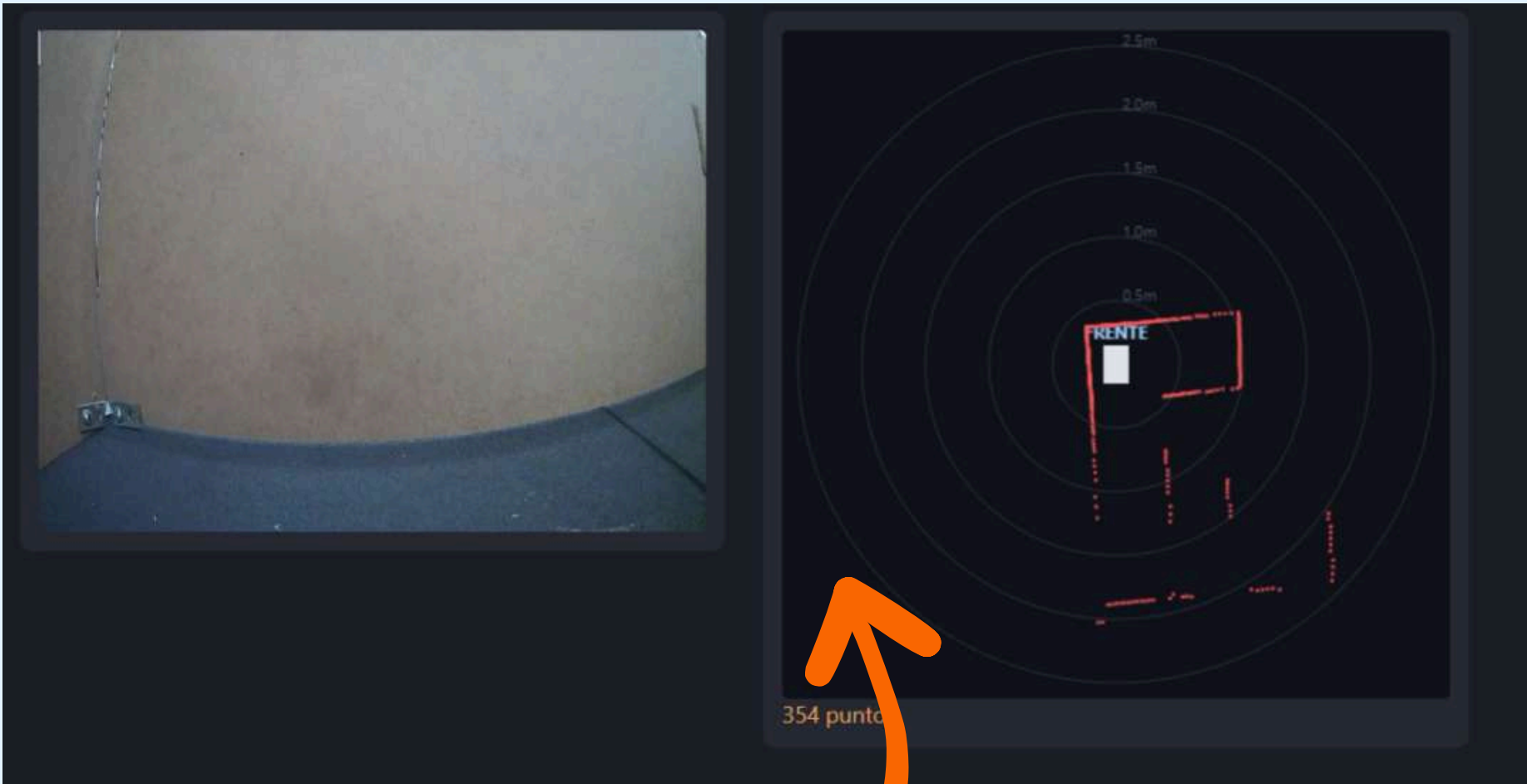
Elemento	Descripción
Pista	360 cm×240 cm; rejilla 6 × 4
Celda	60 cm×60 cm
Inicio y meta	A4 y F1, respectivamente
Robot	Yahboom MicroROS Robot Car, chasis Ackermann
Computador	Raspberry Pi 5
Middleware	ROS 2 Humble
Sensores	LiDAR MS200, cámara frontal y odometría
Ronda 1	Exploración sin conocimiento previo
Ronda 2	Time attack con parámetros optimizados
Regla visual	Detención completa de aproximadamente 3 s ante PARE
Seguridad	Cero contactos con paredes u obstáculos

SISTEMA PROPUESTO

La solución integra LiDAR MS200, cámara frontal y odometría sobre ROS 2 Humble para decidir maniobras en tiempo real.

- percepción geométrica: zonas + recta lateral
- percepción semántica: PARE HSV
- navegación: máquina de estados
- métricas: CSV + eventos en RViz

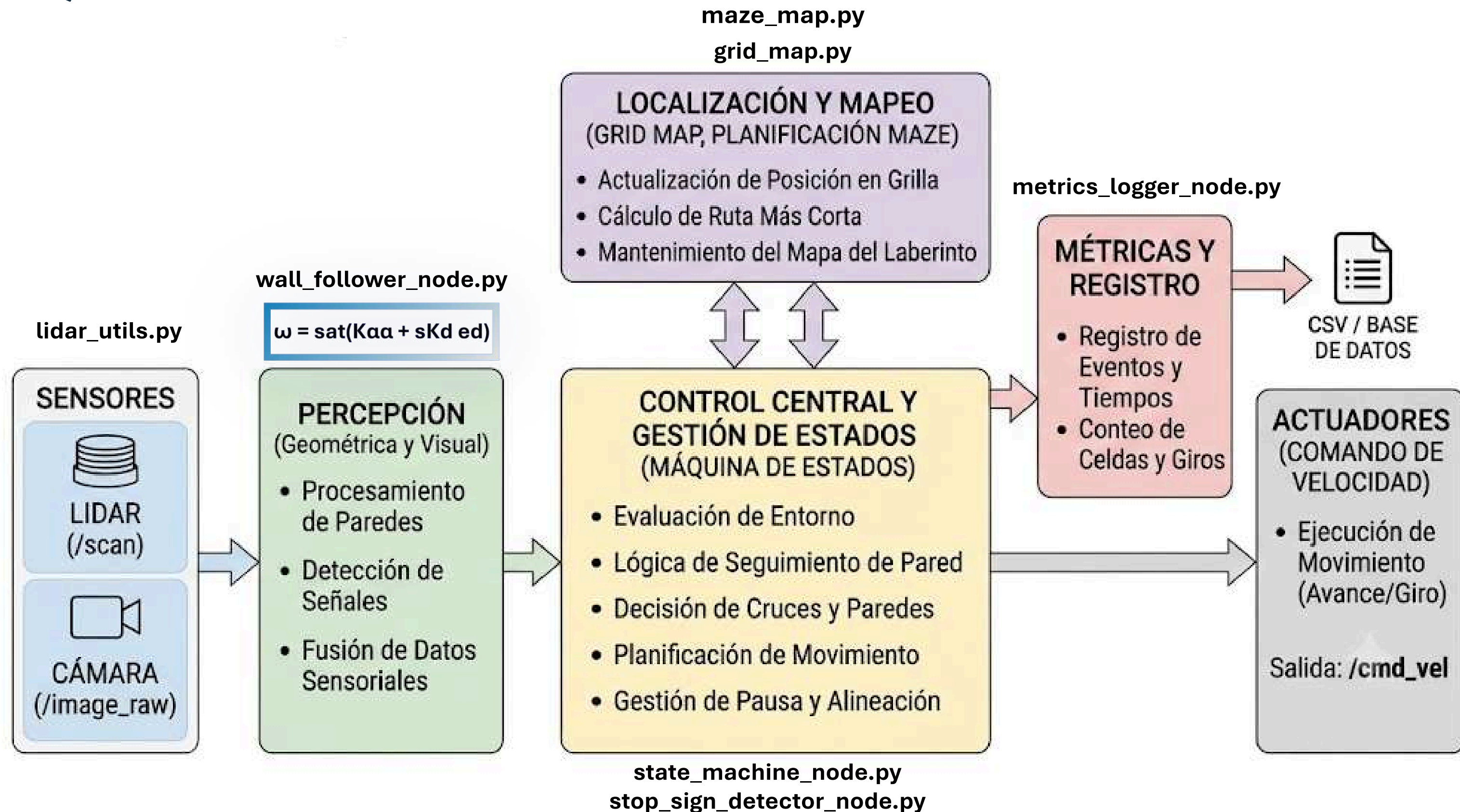
RESPONSABILIDAD DE LOS NODOS PRINCIPALES	
Nodo	Función
lidar_processor	Convierte /scan en distancias por zona y ajusta mediante regresión las paredes izquierda y derecha.
wall_follower	Usa la recta del lado configurado y calcula una sugerencia de velocidad lineal y angular para mantener paralelismo y distancia lateral.
stop_sign_detector	Segmenta rojo en HSV, valida contornos y publica la detección confirmada.
state_machine	Fusiona sensores, decide la maniobra y es el único nodo que publica en /cmd_vel.
metrics_logger	Acumula distancia y eventos; escribe una fila por corrida en formato CSV.



Se define una rejilla de 6 columnas por 4 filas, cuyas celdas están identificadas desde A1 hasta F4. Cada celda mide 60x60 cm. El MicroROS Robot Car inicia en la celda A4 orientado hacia el este y realiza un giro inicial de 90 hacia la izquierda. Cuando completa aproximadamente una celda o detecta una intersección, actualiza su posición lógica y registra las conexiones disponibles en un grafo del laberinto.

ARQUITECTURA

Un solo nodo publica /cmd_vel para evitar órdenes contradictorias.



CODIGO DE LA SOLUCIÓN

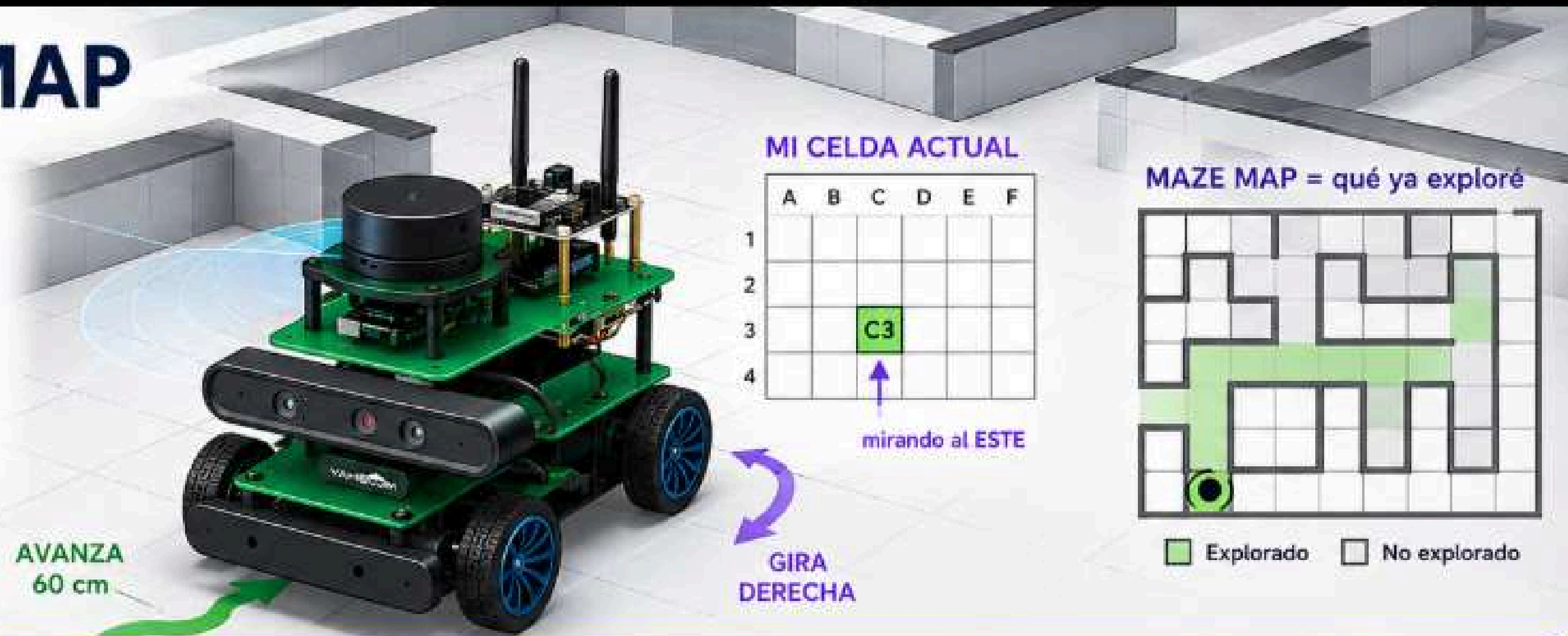
Las funciones principales para la solución del mapeo:

Función	Acción realizada
GridTracker.advance_cell()	Actualiza la celda lógica una posición en el rumbo actual y detecta una salida de la rejilla.
GridTracker.apply_turn()	Actualiza la orientación cardinal después de completar un giro.
MazeMap.record()	Registra aristas entre la celda actual y sus vecinas confirmadas como libres.
MazeMap.shortest_headings()	Ejecuta BFS y devuelve los rumbos absolutos de la ruta más corta conocida.
fit_wall_line()	Convierte puntos LiDAR a coordenadas cartesianas, ajusta una recta y rechaza valores atípicos.
WallFollowerNode._on_zones()	Combina error angular y lateral; genera una sugerencia de velocidad o mantiene el rumbo con odometría si falta pared.
_handle_iniciar()	Registra el inicio en A4 y ordena el giro izquierdo inicial de 90°.
_handle_avanzar_paralelo()	Reenvía el control de pared, mide el avance y detiene el tramo ante una pared frontal.
_handle_chequeo_lado()	Cada 0,12 m, detiene brevemente el robot y confirma si el lado seguido se abrió.
_avanzar_a_siguiente_fase()	Incrementa el conteo de celdas, actualiza el mapa lógico y activa la detección del cruce.
_handle_detectar_cruce()	Clasifica izquierda, frente y derecha mediante consenso de tres de cinco mediciones.
_handle_buscar_pare()	Espera la cámara y mantiene velocidad nula por tres segundos ante una detección confirmada.
_handle_decidir()	Usa el plan BFS si es válido; de lo contrario aplica pared izquierda con preferencia por la celda menos visitada.
_handle_pausa_giro()	Ejecuta un retroceso corto y una pausa para ganar espacio antes del giro.
_handle_girar()	Realiza un arco Ackermann con realimentación del yaw y actualiza la orientación al finalizar.
_handle_alinear()	Compara distancias lateral-delantera y lateral-trasera para quedar paralelo a la pared.
_handle_verificar_meta()	Compara la celda estimada con F1, guarda el mapa y detiene el robot al llegar.
_DetectorColor.procesar()	Confirma o elimina la detección visual mediante histéresis temporal de cuadros consecutivos.
MetricsLoggerNode._on_event()	Cuenta eventos de navegación, seguridad, PARE, callejones y llegada a meta.

Función o método	Archivo donde se encuentra
GridTracker.advance_cell()	grid_map.py
GridTracker.apply_turn()	grid_map.py
MazeMap.record()	maze_map.py
MazeMap.shortest_headings()	maze_map.py
fit_wall_line()	lidar_utils.py
WallFollowerNode._on_zones()	wall_follower_node.py
_handle_iniciar()	state_machine_node.py
_handle_avanzar_paralelo()	state_machine_node.py
_handle_chequeo_lado()	state_machine_node.py
_avanzar_a_siguiente_fase()	state_machine_node.py
_handle_detectar_cruce()	state_machine_node.py
_handle_buscar_pare()	state_machine_node.py
_handle_decidir()	state_machine_node.py
_handle_pausa_giro()	state_machine_node.py
_handle_girar()	state_machine_node.py
_handle_alinear()	state_machine_node.py
_handle_verificar_meta()	state_machine_node.py
_DetectorColor.procesar()	stop_sign_detector_node.py
MetricsLoggerNode._on_event()	metrics_logger_node.py

GRID y MAZE MAP

- El robot navega por seguimiento de pared (wall following) con LIDAR, sin saber su posición absoluta.
- Problema que resuelven GRID y MAZE MAP: si el robot no tiene localización absoluta, ¿cómo sabe en qué celda está y cómo recuerda el laberinto para la segunda vuelta?
- Con dos módulos separados: uno para 'dónde estoy' y otro para 'qué ya exploré'.



GRID (grid_map.py) — ¿en qué celda estoy y hacia dónde miro?

Cada vez que el robot avanza una celda (60 cm) o gira, se actualiza una variable interna — no se mide la posición real del mundo.

```
class GridTracker:  
    col: int  
    row: int  
    heading: str
```

`GridTracker.advance_cell()`: suma el delta del heading actual a (col, row); simula "avanzar 60 cm". Si el resultado cae fuera de la rejilla 6×4, lo recorta al borde y devuelve True como señal de alerta

`GridTracker.apply_turn()`: actualiza el heading según 'DERECHA', 'IZQUIERDA', 'ATRÁS' o 'NINGUNO'

`cell` (propiedad) — devuelve el nombre legible de la celda, ej. "C3"

GRID y MAZE MAP

MAPEO Y RUTA CORTA

GridTracker + MazeMap traducen avance y giros a un grafo de celdas.

6x4

celdas de 60 cm

A4 → F1

inicio y meta

BFS

ruta conocida

JSON

mapa guardado

Si el trazado cambia entre rondas, se descarta el mapa y se vuelve al modo reactivo.

PARÁMETROS DE OPERACIÓN

Valores iniciales usados como punto de partida para calibración en pista.

granprix_params.yaml

Parámetro	Valor
Distancia objetivo a pared	0,12 m
Velocidad lineal base	0,15 m s ⁻¹
Ganancia angular K_{α}	2,0
Ganancia de distancia K_d	2,0
Velocidad angular máxima	0,60 rad s ⁻¹
Frente mínimo del seguidor	0,15 m
Umbral de pared frontal	0,30 m
Umbral de frente libre	0,35 m
Umbral de lado libre	0,35 m
Umbral preventivo frontal	0,18 m
Umbral preventivo lateral	0,11 m
Velocidad lineal de giro	0,05 m s ⁻¹
Velocidad angular de giro	0,80 rad s ⁻¹
Tolerancia de giro	4°
Consenso en cruce	3 de 5 muestras
Tiempo de PARE	3 s



MAZE MAP (maze_map.py) — ¿qué caminos del laberinto ya descubrí?



Grafo no dirigido que representa la conectividad del laberinto: qué celdas están conectadas entre sí



Almacena pares de celdas que están conectadas

```
self._edges = set() # frozenset({(col,row), (col,row)}) por par conectado
```

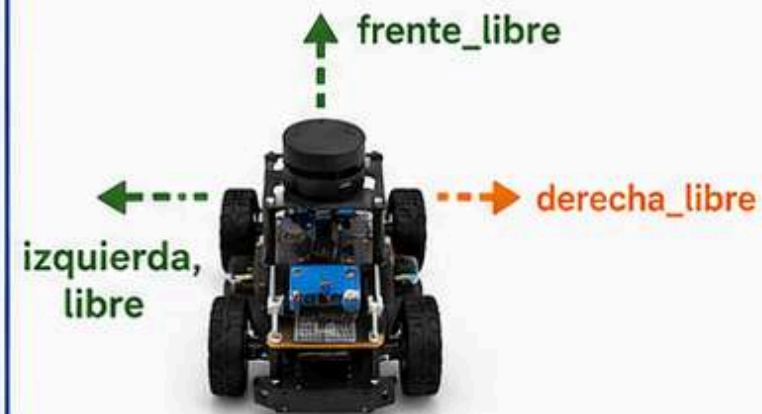


Cada vez que el robot confirma una intersección (parado, con LiDAR), se llama record()

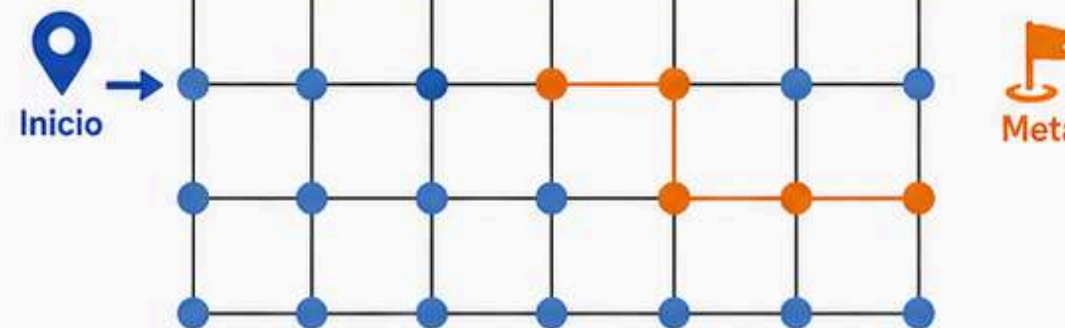


`MazeMap.record(col, row, heading, derecha_libre, frente_libre, izquierda_libre)`

Cuando el robot confirma con el LiDAR, en una intersección, qué direcciones relativas están libres, este método las convierte a direcciones absolutas y agrega las aristas correspondientes al grafo.

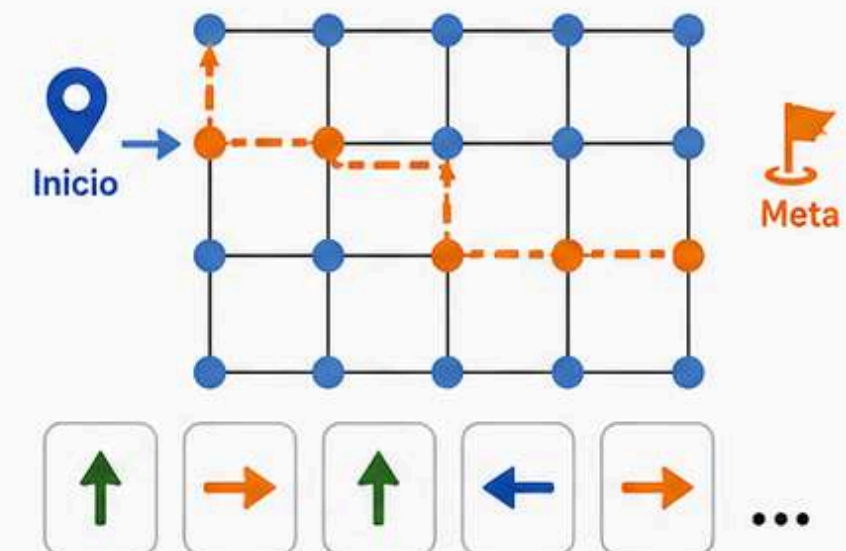


GRAFO DEL LABERINTO (NO DIRIGIDO)



`MazeMap.shortest_headings(start, goal):`
corre BFS

(Breadth-First Search) sobre el grafo acumulado hasta el momento y devuelve la lista de headings a tomar para llegar de start a goal por el camino más corto conocido hasta ahora.



Ronda 1 (exploración): 100% reactiva, regla de mano

Ronda 2 (time attack): se calcula con BFS la ruta más corta conocida desde el inicio. En cada intersección, si lo que el LiDAR confirma libre coincide con el plan

LIDAR UTILS

QUÉ ES

Funciones puras (sin ROS, sin estado) que procesan el arreglo de 360° del LiDAR MS200 y lo convierten en información de navegación: distancia mínima por zona alrededor del robot, y el ajuste de una recta a la pared de un lado.

Lo usa `lidar_processor_node.py` en cada mensaje de `/scan`, y publica el resultado en `/lidar_zones`.

QUÉ PROBLEMA RESUELVE

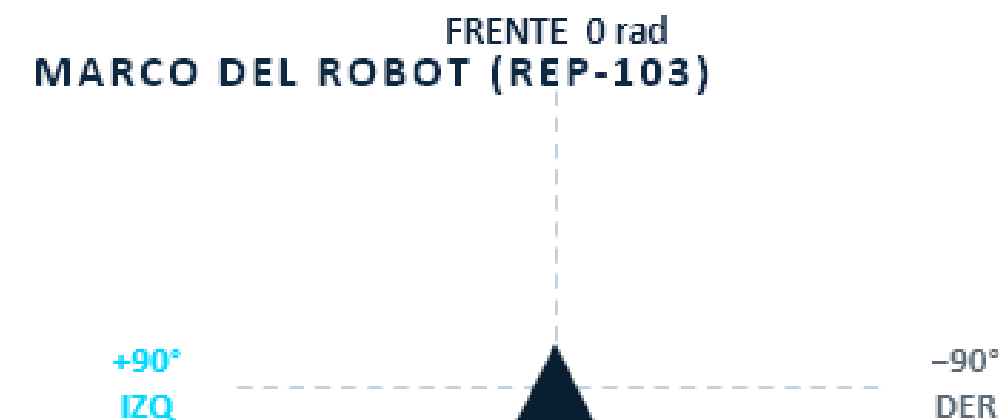
El LiDAR reporta ángulos en el marco del SENSOR, pero está montado en un ángulo que no coincide con el frente real del robot. Este módulo traduce las lecturas al marco del ROBOT (0 rad = frente, +90° izquierda, convención REP-103).



2 parámetros de calibración

`front_offset_rad` qué ángulo del scan es en verdad el frente del robot.

`sign (+1 / -1)` corrige si izquierda/derecha salen invertidas.



LIDAR UTILS

CONTENIDO — 7 PIEZAS EN CADENA

1 **ZoneWindow**
ventana angular $[lo^\circ, hi^\circ]$; envuelve el límite $\pm 180^\circ$ si $lo > hi$

2 **_scan_angles / _robot_frame_angles**
ángulo crudo del scan $\rightarrow$ calibrado al marco del robot

3 **compute_zone_distance**
distancia mínima válida dentro de una ventana

4 **compute_all_zones**
arma todas las zonas (front, left, right...) en una pasada

5 **fit_wall_line**
recta por mínimos cuadrados a la pared de un lado

LA PIEZA MÁS IMPORTANTE

fit_wall_line()

Usa TODOS los puntos válidos de una ventana (ej. -135° a -45°) y ajusta una recta por mínimos cuadrados (`numpy.polyfit`) en el marco del robot — no solo 1-2 puntos, sensibles a ruido.

RECHAZO ITERATIVO DE OUTLIERS

Cerca de una esquina, una pared perpendicular cae dentro de la ventana. Un ajuste simple le da peso indebido: se probó un error falso de hasta $\sim 37^\circ$.

Por eso: ajusta $\rightarrow$ descarta puntos con residuo $> outlier_residual_m \rightarrow$ reajusta, hasta `max_outlier_iter` veces.

Devuelve (**ángulo_rad**, **distancia_m**, **válido**)

WALL FOLLOWER NODE

QUÉ ES

Nodo ROS 2 (WallFollowerNode) que decide la velocidad para mantenerse paralelo y a distancia objetivo de una pared, izquierda o derecha (parámetro lado_seguimiento). Corresponde al estado AVANZAR_PARALELO.

SOLO SUGIERE, NO MUEVE

Publica una velocidad sugerida en /wall_follow/cmd_vel_suggestion. Solo state_machine_node escribe en /cmd_vel de verdad, reenviando la sugerencia mientras dure AVANZAR_PARALELO — evita comandos contradictorios.

/lidar_zones



wall_follower_node



/cmd_vel_suggestion

FÓRMULA DE CONTROL

$$\begin{aligned} \text{angular.z} = & \text{gan_ángulo} \cdot \text{ángulo_pared} \\ & + \text{signo_lado} \cdot \text{gan_distancia} \cdot \\ & (\text{dist_objetivo} - \text{dist_actual}) \end{aligned}$$

Ambas correcciones se SUMAN en el mismo ciclo — nunca alternan.

Término ÁNGULO

mismo signo en ambos lados: paredes izq/der son paralelas, rotar el robot afecta igual a las dos. Sin negar — si se invierte, el lazo diverge en <1s (verificado en simulador).

Término DISTANCIA

cambia de signo (signo_lado: +1 DERECHA / -1 IZQUIERDA) — alejarse de cada lado implica girar en sentido contrario, es un espejo.

WALL FOLLOWER NODE



Pasillo genuinamente abierto

`última distancia > umbral_muy_cerca_m`

Mantiene el RUMBO (heading) con control proporcional (`ganancia_heading`) sobre el yaw de `/odom_raw`, capturado en el instante en que se perdió la pared — en vez de anular la corrección, lo cual dejaría que un sesgo mecánico (Ackermann no centrado) desviara el robot sin control.



Robot demasiado cerca de la pared

`última distancia < umbral_muy_cerca_m`

Gira ACTIVAMENTE lejos de la pared a velocidad angular máxima (con el signo del lado elegido) hasta recuperar lectura válida. Mantener el rumbo aquí es un bug real ya encontrado: si el robot apuntaba algo hacia afuera, se alejaría sin control y nunca volvería.

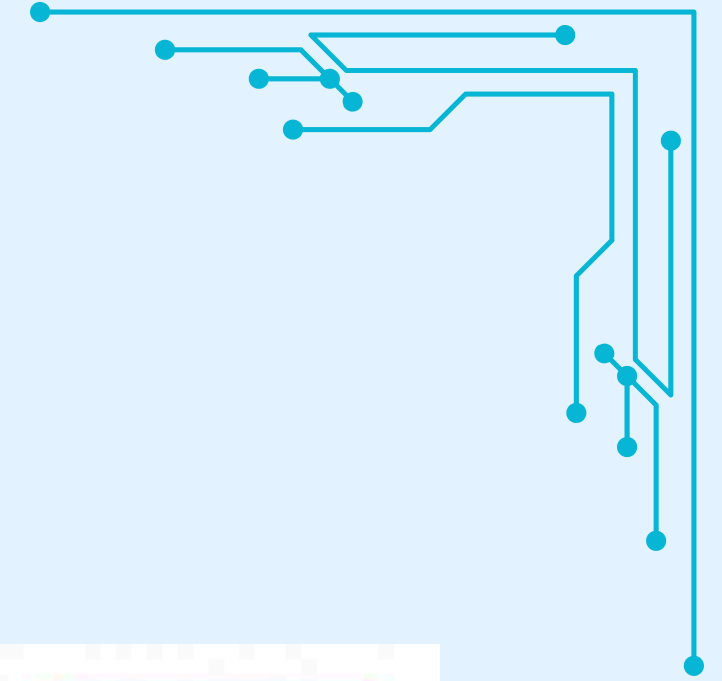
PROTECCIÓN REDUNDANTE

Si hay obstáculo muy cerca al FRENTE (`front_valid` y `front < frente_minimo_seguro_m`), publica velocidad cero. `state_machine_node` también debe reaccionar en su propio ciclo.

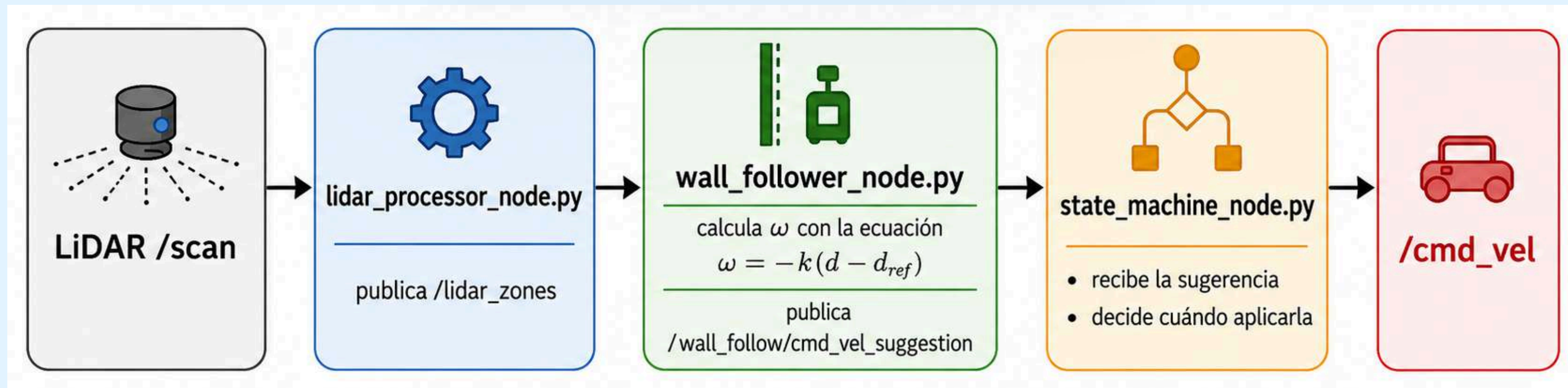
PARÁMETROS AJUSTABLES (YAML)

```
distancia_objetivo_m · velocidad_lineal_mps · ganancia_angulo ·  
ganancia_distancia · ganancia_heading · angular_max_radps ·  
umbral_muy_cerca_m · frente_minimo_seguro_m
```


STATE MACHINE NODE



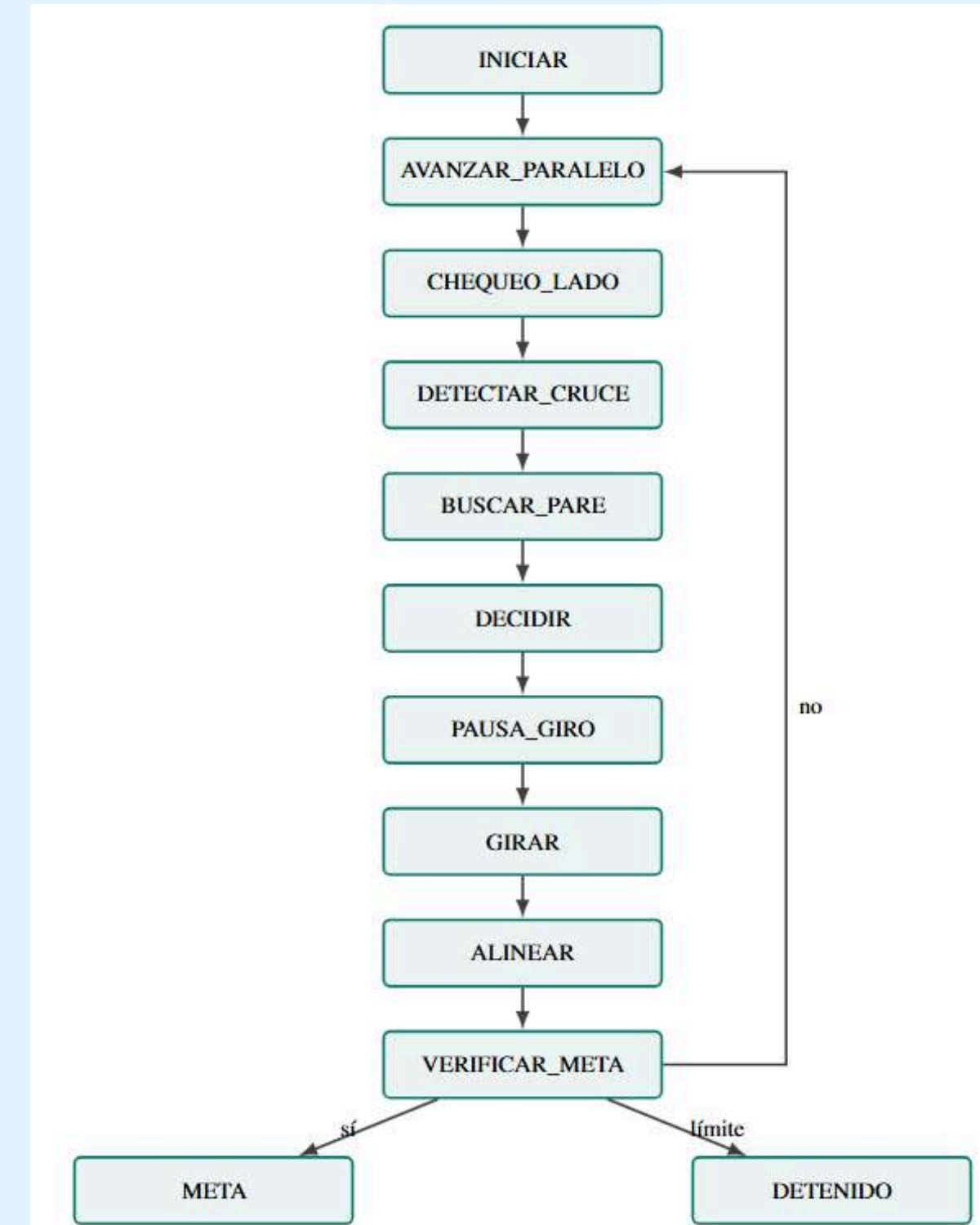
$$\omega = \text{sat}(K\alpha + sKd_{ed})$$



La máquina de estados es, por tanto, el componente que integra LiDAR, cámara, odometría, seguimiento de pared, mapeo por celdas, BFS y seguridad en una sola secuencia de navegación.

Funciones de la clase StateMachineNode

```
self._STATE_HANDLERS = {  
    'INICIAR': self._handle_iniciar,  
    'AVANZAR_PARALELO': self._handle_avanzar_paralelo,  
    'CHEQUEO_LADO': self._handle_chequeo_lado,  
    'DETECTAR_CRUCE': self._handle_detectar_cruce,  
    'BUSCAR_PARE': self._handle_buscar_pare,  
    'DECIDIR': self._handle_decidir,  
    'PAUSA_GIRO': self._handle_pausa_giro,  
    'GIRAR': self._handle_girar,  
    'ALINEAR': self._handle_alinear,  
    'VERIFICAR_META': self._handle_verificar_meta,  
    'META': self._handle_meta,  
    'DETENIDO': self._handle_detenido,  
}
```



1



Iniciación

`_handle_iniciar()`

Establece A4 y orientación norte después del giro inicial.



```
def _handle_iniciar(self):
    self._publish_event(
        EV.INICIO, f'inicio en {self._grid.cell}, heading {self._grid.heading}'
    )

    self._decision_actual = 'IZQUIERDA'
    self._giro_objetivo = self._compute_turn_target(
        self.yaw, 'IZQUIERDA', angulo_rad=self._angulo_giro_inicial_rad
    )
    self._publish_event(EV.GIRO, f'IZQUIERDA (inicial forzado) desde {self._grid.cell}')
    self._publish_twist(Twist())
    self._pausa_giro_start = self.get_clock().now()

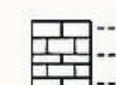
    self._omitir_retroceso_giro = True
    self._set_state('PAUSA_GIRO')
```

Inicializa la exploración, registra el estado del robot y ejecuta un giro forzado de 90° a la izquierda sin retroceder antes de comenzar el recorrido reactivo.

2



Movimiento

`_handle_avanzar_paralelo()`

Sigue la pared y recorre el pasillo.

```
if math.hypot(dx_chk, dy_chk) >= self._distancia_chequeo_pared:

    self._publish_twist(Twist())
    self._contador_lado_libre = 0
    self._pausa_chequeo_start = self.get_clock().now()
    self._set_state('CHEQUEO_LADO')
    return
```

```
frente_cerca = z.front_valid and z.front < self._umbral_frente_pared

if avance >= (self._distancia_celda - self._margen_avance) or frente_cerca:
    self._avanzar_a_siguiente_fase()
    return

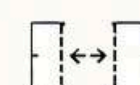
self._publish_twist(self._wall_follow_cmd)
```

Inicializa la referencia de posición y hace avanzar al robot en paralelo a la pared, comprobando periódicamente el lateral y deteniéndose al completar una celda o detectar un obstáculo frontal.

3



Localización de aberturas

`_handle_chequeo_lado()`

Comprueba físicamente si terminó la pared.

```
def _handle_chequeo_lado(self):

    self._publish_twist(Twist())
    elapsed = (self.get_clock().now() - self._pausa_chequeo_start).nanoseconds / 1e9
    if elapsed < self._tiempo_chequeo_pared:
        return

    z = self._zones
    if self._lado == 'IZQUIERDA':
        lado_valid, lado_dist = z.left_valid, z.left
    else:
        lado_valid, lado_dist = z.right_valid, z.right
    lado_libre = bool(lado_valid and lado_dist > self._umbral_lado_libre)
```

```
if lado_libre:
    self._contador_lado_libre += 1
    if self._contador_lado_libre < self._chequeo_pared_confirmaciones_ciclos:
        return
    self._contador_lado_libre = 0
    self._avanzar_a_siguiente_fase()
    return

self._contador_lado_libre = 0

self._chequeo_lado_start_xy = (self._odom_x, self._odom_y)
self._set_state('AVANZAR_PARALELO')
```

Detiene periódicamente el robot para comprobar con el LiDAR si la pared lateral continúa; si detecta una abertura confirmada avanza a la siguiente fase y, si la pared sigue presente, retoma el recorrido paralelo.

4



Actualización lógica



_avanzar_a_siguiente_fase()

Actualiza celda, visitas
y eventos.

```
def _avanzar_a_siguiente_fase(self):
    fuera_de_rango = self._grid.advance_cell()
    celda_actual = (self._grid.col, self._grid.row)
    self._celdas_visitadas[celda_actual] = self._celdas_visitadas.get(celda_actual, 0) + 1
    self._publish_event([
        EV.CELDA_AVANZADA, f'celda {self._grid.cell} ({self._num_celdas})'])
    if fuera_de_rango:
        self._publish_event(
            EV.DERIVA_SOSPECHOSA,
            f'el avance calculado caia fuera de la rejilla 6x4, se recorto a '
            f'{self._grid.cell} -- la celda estimada probablemente ya no coincide '
            'con la posicion fisica real (revisar calibracion de odometria/giro)',
        )
    if self._num_celdas > self._max_celdas:
        self._publish_event(
            EV.TIMEOUT, 'limite de celdas recorridas alcanzado sin llegar a la meta'
        )
    self._guardar_mapa()
    self._terminado = True
    self._set_state('DETENIDO')
    return
```

```
if self._modo_simplificado:
    z = self._zones
    self._derecha_libre = bool(z.right_valid and z.right > self._umbral_lado_libre)
    self._frente_libre = bool(z.front_valid and z.front > self._umbral_frente_libre)
    self._izquierda_libre = bool(z.left_valid and z.left > self._umbral_lado_libre)
    self._derecha_libre, self._frente_libre, self._izquierda_libre = (
        self._filtrar_fuera_de_rejilla(
            self._derecha_libre, self._frente_libre, self._izquierda_libre))
    self._maze_map.record(
        self._grid.col, self._grid.row, self._grid.heading,
        self._derecha_libre, self._frente_libre, self._izquierda_libre)
    self._set_state('DECIDIR')
else:
    self._set_state('DETECTAR_CRUCE')
```

Registra el avance a una nueva celda, actualiza el mapa y las visitas, controla posibles errores o límites de recorrido y dirige al robot hacia la decisión inmediata o la detección detallada del cruce.

5



Percepción del cruce



_handle_detectar_cruce()

Determina caminos libres
y construye el grafo.

```
def _handle_detectar_cruce(self):
    self._publish_twist(Twist())

    if self._cruce_muestras is None:
        self._cruce_muestras = {'right': [], 'front': [], 'left': []}

    z = self._zones
    self._cruce_muestras['right'].append(
        bool(z.right_valid and z.right > self._umbral_lado_libre)
    )
    self._cruce_muestras['front'].append(
        bool(z.front_valid and z.front > self._umbral_frente_libre)
    )
    self._cruce_muestras['left'].append(
        bool(z.left_valid and z.left > self._umbral_lado_libre)
    )

    if len(self._cruce_muestras['right']) < self._muestras_confirmacion:
        return
```

```
def _handle_detectar_cruce(self):
    def consenso(muestras):
        return sum(muestras) >= self._consenso_minimo

    self._derecha_libre = consenso(self._cruce_muestras['right'])
    self._frente_libre = consenso(self._cruce_muestras['front'])
    self._izquierda_libre = consenso(self._cruce_muestras['left'])
    self._cruce_muestras = None

    self._derecha_libre, self._frente_libre, self._izquierda_libre = (
        self._filtrar_fuera_de_rejilla(
            self._derecha_libre, self._frente_libre, self._izquierda_libre
        )
    )
    self._maze_map.record(
        self._grid.col, self._grid.row, self._grid.heading,
        self._derecha_libre, self._frente_libre, self._izquierda_libre,
    )
```

```
self._publish_event(
    EV.CRUCE,
    f'derecha={self._derecha_libre} frente={self._frente_libre} '
    f'izquierda={self._izquierda_libre}',
)
self._publish_marker('CRUCE', f'cruce {self._grid.cell}')

self._buscar_pare_start = self.get_clock().now()
self._pare_hold_start = None
self._set_state('BUSCAR_PARE')
```

Detiene el robot en cada intersección, recopila varias lecturas del LiDAR y aplica un criterio de consenso para confirmar qué direcciones están libres, descarta movimientos fuera de la rejilla, actualiza el mapa del laberinto. Cuando _handle_chequeo_lado() confirmó que la pared lateral terminó y apareció una abertura.

6



Regla de tránsito

`_handle_buscar_pare()`

3s

Detiene tres segundos ante PARE.

```
def _handle_buscar_pare(self):
    self._publish_twist(Twist())
    if not self._usar_camara:
        self._set_state('DECIDIR')
        return
    cell = self._grid.cell
    if self._pare_hold_start is not None:
        elapsed = (self.get_clock().now() - self._pare_hold_start).nanoseconds / 1e9
        self._despachar_pitido_pendiente(elapsed)
        if elapsed >= self._tiempo_pare:
            self._celdas_pare_respetadas.add(cell)
            self._publish_event(EV.PARE_RESPETADO, f'PARE respetado en {cell}')
            self._publish_marker('PARE_RESPETADO', f'PARE ok {cell}')
            self._set_state('DECIDIR')
    return
```

```
if self._pare_activo and cell not in self._celdas_pare_respetadas:
    self._publish_event(EV.PARE_DETECTADO, f'senal PARE detectada en {cell}')
    self._publish_marker('PARE_DETECTADO', f'PARE {cell}')
    self._emitir_pitido_pare()
    self._pare_hold_start = self.get_clock().now()
    return

elapsed_settle = (self.get_clock().now() - self._buscar_pare_start).nanoseconds / 1e9
if elapsed_settle >= self._tiempo_espera_camara:
    self._set_state('DECIDIR')
```

Verifica con la cámara la presencia de una señal de PARE, emite una alerta y mantiene el robot detenido durante tres segundos antes de continuar con la decisión de movimiento.

7



Planificación

`_handle_decidir()`

Selecciona BFS o navegación reactiva.

```
def _handle_decidir(self):
    direction = self._direccion_planificada()
    if direction is None:
        if self._lado == 'IZQUIERDA':
            orden = (
                ('IZQUIERDA', self._izquierda_libre),
                ('NINGUNO', self._frente_libre),
                ('DERECHA', self._derecha_libre),
            )
        else:
            orden = (
                ('DERECHA', self._derecha_libre),
                ('NINGUNO', self._frente_libre),
                ('IZQUIERDA', self._izquierda_libre),
            )
        candidatos = [d for d, libre in orden if libre]
        direction = self._elegir_direccion_sin_loop(candidatos)
```

```
if direction is None:
    direction = 'ATRAS'
    self._publish_event(EV.DEAD_END, f'callejon sin salida en {self._grid.cell}')
self._decision_actual = direction
if direction == 'NINGUNO':
    if self._modo_simplificado:
        self._begin_avanzar_paralelo()
        self._set_state('AVANZAR_PARALELO')
    else:
        self._alinear_start = None
        self._set_state('ALINEAR')
    return
self._giro_objetivo = self._compute_turn_target(self._yaw, direction)
self._publish_event(EV.GIRO, f'{direction} desde {self._grid.cell}')
self._publish_twist(Twist())
self._pausa_giro_start = self.get_clock().now()
self._set_state('PAUSA_GIRO')
```

Selecciona la dirección que seguirá el robot; primero intenta ejecutar la ruta más corta calculada mediante BFS y, si el plan no es válido, aplica la regla de seguimiento de pared priorizando las celdas menos visitadas para evitar ciclos.

8

←--- Preparación del giro

→ `_handle_pausa_giro()`

Retrocede y crea margen de seguridad.

```
def _handle_pausa_giro(self):  
    elapsed = (self.get_clock().now() - self._pausa_giro_start).nanoseconds / 1e9  
    tiempo_retroceso = 0.0 if self._omitir_retroceso_giro else self._tiempo_retroceso_giro  
  
    if elapsed < tiempo_retroceso:  
        cmd = Twist()  
        cmd.linear.x = -self._v_retroceso_giro  
        self._publish_twist(cmd)  
        return  
  
    self._publish_twist(Twist())  
    if elapsed >= self._tiempo_pausa_antes_girar:  
        self._omitir_retroceso_giro = False  
        self._girar_start = self.get_clock().now()  
        self._set_state('GIRAR')
```

Hace retroceder ligeramente al robot para separarlo de la pared o esquina y luego lo mantiene detenido antes de iniciar el giro.

9



Ejecución del giro

→ `_handle_girar()`

Realiza el arco y actualiza la orientación.

```
def _handle_girar(self):  
    error = angle_diff(self._giro_objetivo, self._yaw)  
  
    elapsed = (self.get_clock().now() - self._girar_start).nanoseconds / 1e9  
    if elapsed >= self._tiempo_max_girar:  
        self._publish_event(  
            EV.DERIVA_SOSPECHOSA,  
            f'GIRAR supero tiempo_max_girar_s ({self._tiempo_max_girar:.1f}s) sin '  
            f'completar el angulo objetivo (error={math.degrees(error):.1f} deg) -- '  
            'se fuerza a terminar el giro; revisar calibracion de odometria/giro.',  
        )  
        self._publish_twist(Twist())  
        self._grid.apply_turn(self._decision_actual)  
        self._alineal_start = None  
        self._set_state('ALINEAR')  
        return  
  
    if abs(error) <= self._tolerancia_giro_rad:  
        self._publish_twist(Twist())  
        self._grid.apply_turn(self._decision_actual)  
  
        self._alineal_start = None  
        self._set_state('ALINEAR')  
        return  
  
    cmd = Twist()  
    cmd.linear.x = self._v_giro_lineal  
    cmd.angular.z = self._v_giro_angular if error > 0.0 else -self._v_giro_angular  
    self._publish_twist(cmd)
```

Ejecuta un giro en arco compatible con la dirección Ackermann, compara continuamente el ángulo actual con el objetivo mediante la odometría y actualiza la orientación lógica al completar el giro.



```
def _handle_alinear(self):
    if self._alinear_start is None:
        self._alinear_start = self.get_clock().now()

    z = self._zones
    if self._lado == 'DERECHA':
        front_valido, rear_valido = z.right_front_valid, z.right_rear_valid
        front, rear = z.right_front, z.right_rear
    else:
        front_valido, rear_valido = z.left_front_valid, z.left_rear_valid
        front, rear = z.left_front, z.left_rear

    if not (front_valido and rear_valido):

        self._alinear_start = None
        self._set_state('VERIFICAR_META')
        return
```

```
error_angulo = front - rear
elapsed = (self.get_clock().now() - self._alinear_start).nanoseconds / 1e9

if abs(error_angulo) <= self._tolerancia_alineacion or elapsed >= self._tiempo_max_alinear:
    self._publish_twist(Twist())
    self._alinear_start = None
    self._set_state('VERIFICAR_META')
    return

cmd = Twist()
cmd.linear.x = self._v_alinear_lineal
if self._lado == 'DERECHA':
    cmd.angular.z = -self._v_alinear_angular if error_angulo > 0.0 else self._v_alinear_angular
else:
    cmd.angular.z = self._v_alinear_angular if error_angulo > 0.0 else -self._v_alinear_angular
self._publish_twist(cmd)
```

Compara las distancias medidas por el LiDAR en la parte delantera y trasera del lateral seguido, corrigiendo la trayectoria hasta que el robot quede paralelo a la pared.



```
def _handle_verificar_meta(self):
    if self._grid.cell == self._celda_meta:
        self._publish_twist(Twist())
        self._publish_event(EV.META, f'meta alcanzada en {self._grid.cell}')
        self._guardar_mapa()
        self._terminado = True
        self._set_state('META')
        return
```

Comprueba si la celda estimada coincide con la meta F1; si llega, detiene el robot y guarda el mapa, y si no, reinicia el avance paralelo.

STOP SING DETECTOR NODE

Detecta ambos carteles: PARE (rojo) y META (verde, opcional) usando la cámara del robot

- Segmenta por color en HSV: convierte el frame a espacio HSV para aislar el color de interés
- Filtra por forma del blob: área, relación de aspecto y solidez descartan ruido y reflejos
- Exige confirmación temporal: varios frames consecutivos antes de avisar y varios para retirarla

Publica:

- /pare_detectado (Bool) true si el cartel PARE está confirmado
- /meta_detectado (Bool) opcional, true si META está confirmado



```
self._pare = _DetectorColor(
    rango1_min=np.array(gp('rango1_min'), dtype=np.uint8),
    rango1_max=np.array(gp('rango1_max'), dtype=np.uint8),
    rango2_min=np.array(gp('rango2_min'), dtype=np.uint8),
    rango2_max=np.array(gp('rango2_max'), dtype=np.uint8),
    area_min=float(gp('area_minima_px')),
    area_max=float(gp('area_maxima_px')),
    aspecto_min=float(gp('relacion_aspecto_min')),
    aspecto_max=float(gp('relacion_aspecto_max')),
    solidez_min=float(gp('solidez_minima')),
    frames_confirmacion=int(gp('frames_confirmacion')),
    frames_perdida=int(gp('frames_perdida')),
    exigir_centro=True,
    banda_central_frac=float(gp('banda_central_frac')),
)
# El detector de META se instancia siempre (no cuesta nada en
```

```
self.confirmado = False

def _mascara(self, hsv):
    mask = cv2.inRange(hsv, self.rango1_min, self.rango1_max)
    if self.rango2_min is not None:
        mask = cv2.bitwise_or(mask, cv2.inRange(hsv, self.rango2_min, self.rango2_max))
    kernel = np.ones((5, 5), np.uint8)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
    return mask
```

El rojo (PARE) necesita dos rangos de HSV porque el matiz rojo cruza el 0/179.

MORPH_OPEN limpia ruido pequeño; MORPH_CLOSE rellena huecos dentro del blob.

STOP SING DETECTOR NODE

`self.consec_detect = 0`

`self.consec_lost = 0`

`self.confirmado = False`

- **consec_detect**: cuenta los frames consecutivos en que el cartel Sí fue detectado.
- **consec_lost**: cuenta los frames consecutivos en que el cartel NO fue detectado.
- **confirmado**: bandera True/False que indica si el cartel ya quedó confirmado, una vez que **consec_detect** supera el umbral de **frames_confirmacion**.



```
def procesar(self, hsv, y_offset, ancho_frame):
    mask = self._mascara(hsv)
    mask = self._quitar_componentes_pequenos(mask)
    mejor = self._mejor_blob(mask, y_offset, ancho_frame)

    if mejor is not None:
        self.consec_detect += 1
        self.consec_lost = 0
    else:
        self.consec_lost += 1
        self.consec_detect = 0

    if not self.confirmado and self.consec_detect >= self.frames_confirmacion:
        self.confirmado = True
    elif self.confirmado and self.consec_lost >= self.frames_perdida:
        self.confirmado = False

    bbox = mejor[1] if mejor is not None else None
    return self.confirmado, bbox
```

```
def _mejor_blob(self, mask, y_offset, ancho_frame):
    cx_frame = ancho_frame / 2.0
    tol_px = self.banda_central_frac * ancho_frame
    contornos, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    mejor = None
    for cnt in contornos:
        area = cv2.contourArea(cnt)
        if area < self.area_min or area > self.area_max:
            continue

        x, y, w, h = cv2.boundingRect(cnt)
        if h == 0:
            continue
        if self.exigir_centro and abs((x + w / 2.0) - cx_frame) > tol_px:
            continue

        aspecto = w / float(h)
        if not (self.aspecto_min <= aspecto <= self.aspecto_max):
            continue

        hull = cv2.convexHull(cnt)
        area_hull = cv2.contourArea(hull)
        if area_hull < 1e-3 or area / area_hull < self.solidéz_min:
            continue

        if mejor is None or area > mejor[0]:
            mejor = (area, (x, y + y_offset, w, h))

    return mejor # None o (area, bbox)
```



METRICS LOGGER NODE

- Escucha /robot_event para contar colisiones, PARE y callejones sin salida, sin acoplarse a la lógica de navegación.
- Escucha /odom_raw para acumular la distancia recorrida.
- Al recibir META (o TIMEOUT, o si se apaga sin llegar) escribe una fila en metricas_granprix.csv.
- Cada vez que llega una lectura de odometría, calcula la distancia entre el punto actual y el anterior (math.hypot = distancia euclidiana) y la va sumando.
- Convierte de metros a centímetros (* 100.0).

```
def _on_odom(self, msg: Odometry) -> None:
    x = msg.pose.pose.position.x
    y = msg.pose.pose.position.y
    if self._prev_xy is not None:
        dx = x - self._prev_xy[0]
        dy = y - self._prev_xy[1]
        self._long_ruta_cm += math.hypot(dx, dy) * 100.0
    self._prev_xy = (x, y)
```

```
class MetricsLoggerNode(Node):

    def __init__(self):
        super().__init__('metrics_logger')

        self.declare_parameter('event_topic', '/robot_event')
        self.declare_parameter('odom_topic', '/odom_raw')
        self.declare_parameter('csv_path', '~/capytown_resultados/metricas_granprix.csv')
        self.declare_parameter('ronda', 1)
        self.declare_parameter('pare_reales', 3)
        self.declare_parameter('long_optima_cm', 480.0)

        self._csv_path = os.path.expanduser(self.get_parameter('csv_path').value)
        self._ronda = int(self.get_parameter('ronda').value)
        self._pare_reales = int(self.get_parameter('pare_reales').value)
        self._long_optima_cm = float(self.get_parameter('long_optima_cm').value)

        self._start_time = self.get_clock().now()
        self._prev_xy = None
        self._long_ruta_cm = 0.0

        self._colisiones = 0
        self._pare_detectados = 0
        self._pare_respetados = 0
        self._pare_falsos = 0
        self._dead_ends = 0
        self._finalizado = False

        self.create_subscription(RobotEvent, self.get_parameter('event_topic').value, self._on_event, 10)
        self.create_subscription(Odometry, self.get_parameter('odom_topic').value, self._on_odom, 10)

        self.get_logger().info(
            f'metrics_logger listo: ronda={self._ronda}, csv={self._csv_path}'
        )
```

METRICS LOGGER NODE

```
def _on_event(self, msg: RobotEvent) -> None:
    tipo = msg.tipo

    if tipo == EV.COLISION:
        self._colisiones += 1
    elif tipo == EV.PARE_DETECTADO:
        self._pare_detectados += 1
    elif tipo == EV.PARE_RESPETADO:
        self._pare_respetados += 1
    elif tipo == EV.PARE_FALSO:
        self._pare_falsos += 1
    elif tipo == EV.DEAD_END:
        self._dead_ends += 1
    elif tipo == EV.META:
        self._finalizar(llego_meta=True)
    elif tipo == EV.TIMEOUT:
        self._finalizar(llego_meta=False)
```

```
def _escribir_fila_csv(self, fila: dict) -> None:
    os.makedirs(os.path.dirname(self._csv_path), exist_ok=True)
    existe = os.path.isfile(self._csv_path)
    with open(self._csv_path, 'a', newline='', encoding='utf-8') as f:
        writer = csv.DictWriter(f, fieldnames=_CSV_FIELDS)
        if not existe:
            writer.writeheader()
        writer.writerow(fila)
```

Crea la carpeta si no existe (os.makedirs).

Si el archivo no existe aún, escribe primero el encabezado (writeheader).

Cada corrida agrega una fila nueva (modo 'a' = append), acumulando el historial de todas las rondas en el mismo CSV.

- Cada tipo de evento incrementa su propio contador.
- $\text{eficiencia} = \text{distancia óptima} / \text{distancia real recorrida}$ (mientras más cerca de 1, más eficiente fue la ruta).
- Al llegar META o TIMEOUT, se arma la fila y se escribe al CSV (una sola vez, protegido por self._finalizado).



RESULTADOS OBTENIDOS



80%

tasa global de éxito

63.24%

precisión PARE

86%

recall PARE

3

colisiones

2

corridas incompletas

40

PARE respetados



Ronda 1

Ronda 2

Tiempo



91.62s
64.48s

Longitud

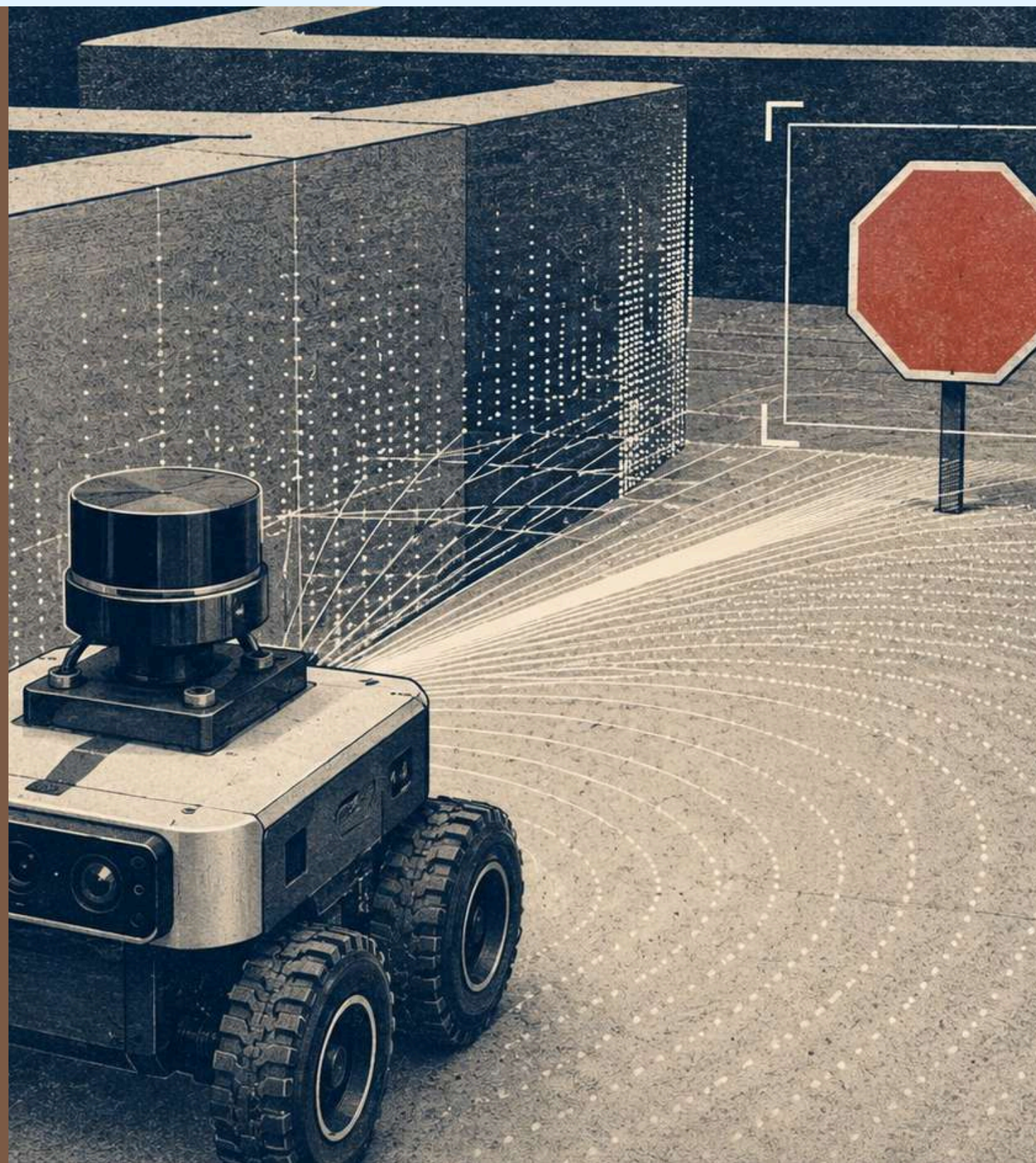


7.85m
5.42m

Eficiencia



0.611
0.886



CONCLUSIONES Y MEJORAS

El sistema completa la mayoría de recorridos, pero aún no es completamente confiable.

Logro: arquitectura modular con fusión LiDAR–
cámara–odometría.

Punto crítico: sensibilidad a iluminación, esquinas y
deriva de odometría.

Próximo paso: localización robusta, mapa de
ocupación, A* y detección profunda.

**Una mejora válida no solo reduce tiempo: también debe respetar PARE,
evitar colisiones y elevar eficiencia.**